

Introduction to Machine Learning. Regression

Rita Almeida

Autumn 2025

```
library('tidyverse')
library("gridExtra")
theme_set(theme_bw())
set.seed(12383)
```

Prediction

Generating data and fitting a linear model

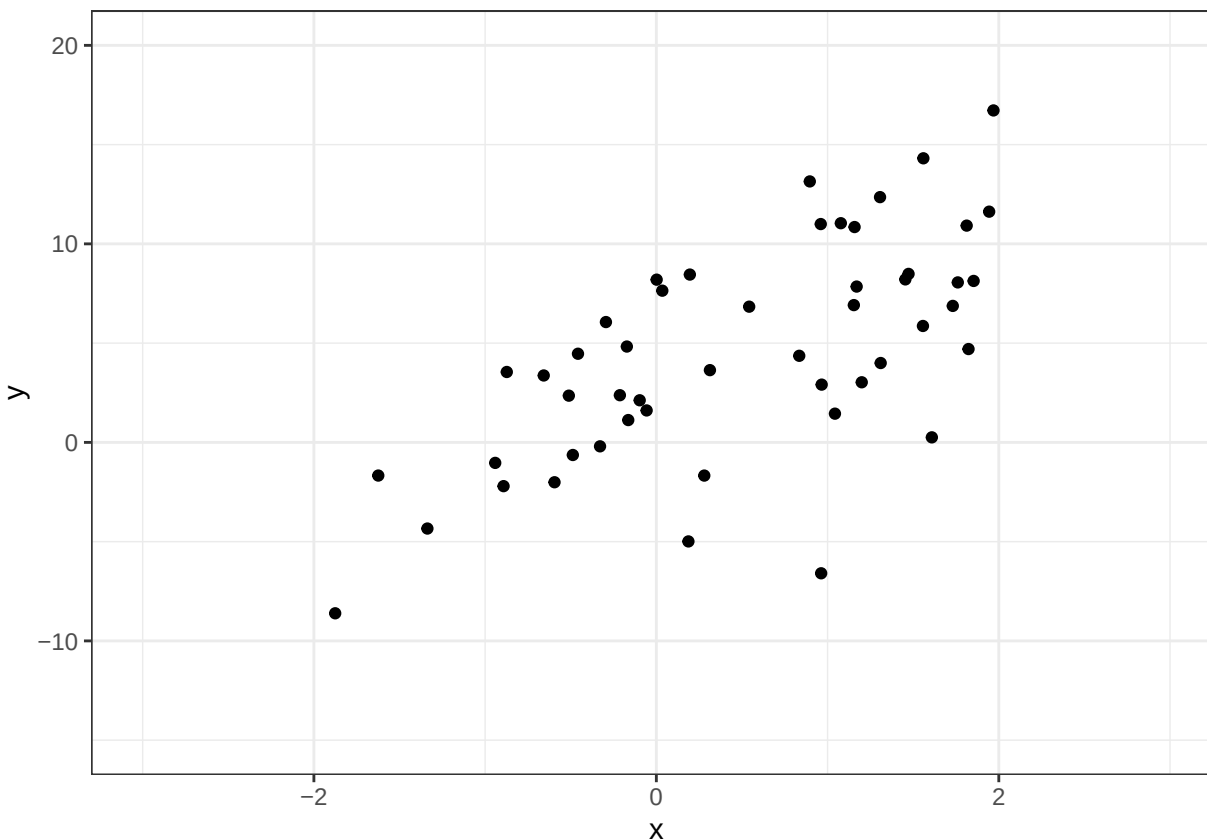
In this section we fit a linear model to a simulated training data set. The model is then used to make predictions on another simulated data set, the test data set.

We start by generating the training data *dTrain1* consisting of 50 random pairs (x,y), with *x* restricted to be between -2 and 2. The relation between *x* and *y* is $y = 2 + 3x + e$, where *e* is normally distributed with mean 0 and variance 4.

```
dTrain1 <- tibble(x = runif(50, -2, 2),
  y = 2 + 3 * x + rnorm(length(x), 0, 4))
```

The data points can be visualized using a scatter plot.

```
ggplot(dTrain1, aes(x, y)) +
  geom_point() + xlim(-3,3)+ ylim(-15,20)
```



Next we fit a linear model $y = \beta_0 + \beta_1 x + \epsilon$ to the data, using least squares, with the *lm* command you have used before. We use the *summary* command to look at the results.

```
m1 <- lm(y ~ x, dTrain1)
summary(m1)
```

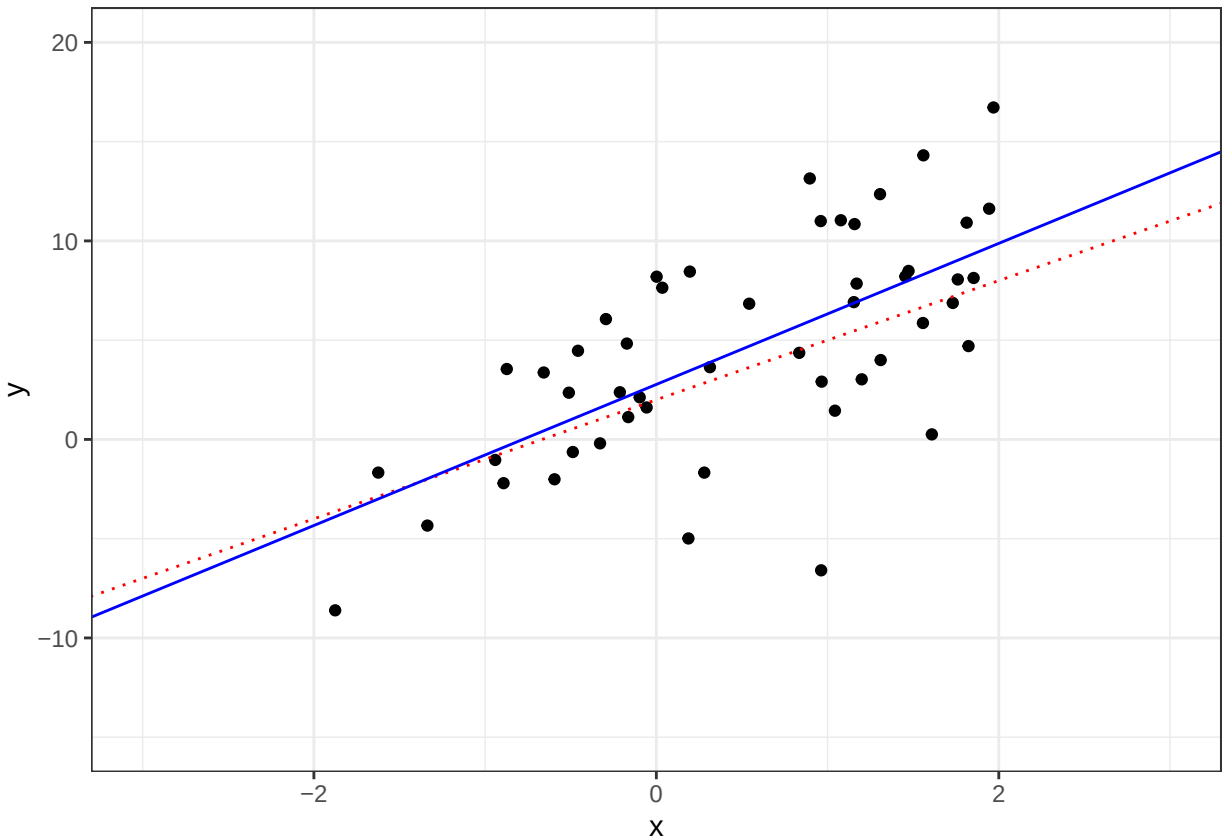
```
##
## Call:
## lm(formula = y ~ x, data = dTrain1)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -12.7825  -2.2805  -0.0973   3.2252   7.1921
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    2.7691     0.6611   4.189 0.000119 ***
## x              3.5528     0.5858   6.065 1.99e-07 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 4.209 on 48 degrees of freedom
## Multiple R-squared:  0.4339, Adjusted R-squared:  0.4221
## F-statistic: 36.79 on 1 and 48 DF,  p-value: 1.993e-07
```

Finally we plot: the data points; the true relation between x and y in red, dotted line; the population regression line we estimated above in blue, solid line. Note that b_0 and b_1 are the estimated intercept and

slope from model *m1*.

```
coefs1 <- coefficients(m1)
b0 <- coefs1[1] # Intercept
b1 <- coefs1[2] # Slope

ggplot(dTrain1, aes(x, y)) +
  geom_point() + xlim(-3,3) + ylim(-15,20) +
  geom_abline(intercept = 2, slope = 3, color="red", linetype="dotted") +
  geom_abline(intercept = b0, slope = b1, color="blue")
```



Making predictions for a new data set

When the aim of estimating a function $f(X)$ is prediction, one uses the estimated $\hat{f}$ to predict the values $\hat{y}$. For this example, we start by generating a new data set with 10 observations, the test data set *dTest1*, in the same way we generated the training set.

```
dTest1 <- tibble(x = runif(10, -2, 2),
  y = 2 + 3 * x + rnorm(length(x), 0, 4))
```

Next we calculate for each *x* in *dTest1* the predicted values of *y* (named *pred* in the code). We use the formula for *f* with the obtained values for the intercept and slope estimated from the training data. There is a command in R to obtain the predicted values given a model (*predict.lm*, see below) but here we write the formula explicitly for clarity. The predicted values are added as a column to the data frame *dTest1*.

```
dTest1$pred <- b0+b1*dTest1$x
```

The prediction error for each *y* is simply the difference between the actual *y* and the predicted value. Below

we add a column to *dTest1* containing this prediction error, and another column containing the square of the prediction error.

```
dTest1$error <- dTest1$y-dTest1$pred
dTest1$sqerror <- (dTest1$y-dTest1$pred)^2
dTest1
```

```
## # A tibble: 10 x 5
##       x       y   pred error sqerror
##   <dbl> <dbl> <dbl> <dbl>   <dbl>
## 1  0.444  2.44   4.35  -1.90     3.61
## 2 -1.69   2.22  -3.24   5.46    29.8
## 3 -0.881 -4.54  -0.361 -4.17    17.4
## 4 -1.58   4.07  -2.84   6.90    47.7
## 5  0.462  0.136  4.41  -4.27    18.3
## 6 -1.23   2.12  -1.61   3.72    13.9
## 7  1.73   8.19   8.92  -0.729   0.532
## 8  1.98   7.07   9.81  -2.74     7.52
## 9 -0.243 -5.75   1.91  -7.66    58.7
## 10 0.114 -3.04   3.18  -6.21    38.6
```

The mean of the squared errors is the test **mean squared error** MSE, and can be easily calculated:

```
mean(dTest1$sqerror)
```

```
## [1] 23.59479
```

For comparison we also print the value of the training MSE. Below we also show how to obtain the test MSE in a more succinct way.

```
MSE.Train1 = mean(m1$residuals^2)
MSE.Test1 = mean((dTest1$y - predict.lm(m1, newdata=dTest1)) ^ 2)

print(MSE.Train1)
```

```
## [1] 17.00541
```

```
print(MSE.Test1)
```

```
## [1] 23.59479
```

Variance of the estimators

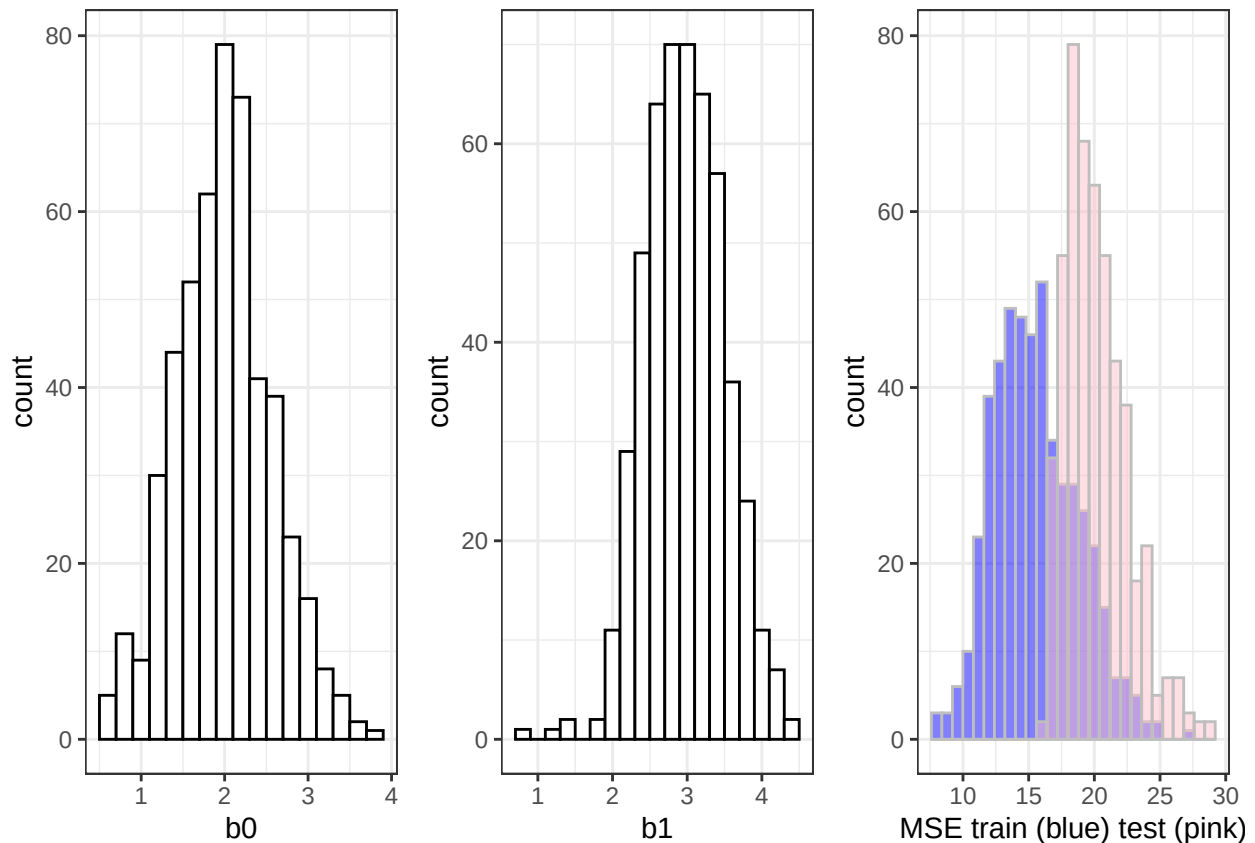
Let us draw 500 samples of training data in the same way as before. For each sample we fit a linear model and estimated intercept, slope, training MSE and test MSE. We will not generate new test data. We will keep using the same test data generated before.

```
for (i in 1:500) {
  dTrain <- tibble(x = runif(50, -2, 2),
                  y = 2 + 3 * x + rnorm(length(x), 0, 4))
  m <- lm(y ~ x, dTrain)
  coefs <- coefficients(m)
  est<-tibble(b0=coefs[1], b1=coefs[2],MSE.Train=mean(m$residuals^2),
              MSE.Test=mean((dTest1$y - predict.lm(m, newdata=dTest1)) ^ 2))
  if (i==1) {est1<-est}
  est1 <- rbind(est1, est )
}
```

These 500 samples and model fits can be used to look at the variance in the parameter estimates, and the means of the MSEs.

We can first check the distribution of the quantities we calculated in the 500 samples.

```
p1 <- ggplot(est1, aes(x=b0)) +  
  geom_histogram(binwidth=.2, colour="black", fill="white")  
p2 <- ggplot(est1, aes(x=b1)) +  
  geom_histogram(binwidth=.2, colour="black", fill="white")  
p3 <- ggplot()+  
  geom_histogram(aes(x = est1$MSE.Train), alpha = 0.5, binwidth=.8, colour="gray", fill="blue")+  
  geom_histogram(aes(x = est1$MSE.Test), alpha = 0.5, binwidth=.8, colour="gray", fill="pink") +  
  xlab("MSE train (blue) test (pink)")  
grid.arrange(p1, p2,p3,ncol=3)
```



And then calculate the variances and means.

```
var(est1$b0)  
## [1] 0.3268958  
var(est1$b1)  
## [1] 0.2790547  
mean(est1$MSE.Train)  
## [1] 15.55686
```

```
mean(est1$MSE.Test)
```

```
## [1] 20.17828
```

Exercise

But how does the variance of the estimators and the train and test MSEs change when smaller samples are used to fit the model? To gain intuition on this question repeat the steps just above, but generate training sets with 10 data points. Continue to use the same test data already generated. Compare the variances of the estimates and the means of the MSEs obtained using the larger samples (50 data points) and the smaller samples (10 data points). What do you conclude?

```
# write your solution here
```

Exercise

We have just looked at the consequences of using different training set sizes. What happens if one uses different test set sizes? Consider a single model that you fitted using a training data set with 50 observations. Use this model to generate predictions for test data sets. Start by making predictions for 500 test data sets with 10 observations each. Then make new predictions for 500 test data sets with 50 observations each. Comment on the effect the size of the test data set has on the mean and variance of the test MSE.

```
# write your solution here
```

Comparing models

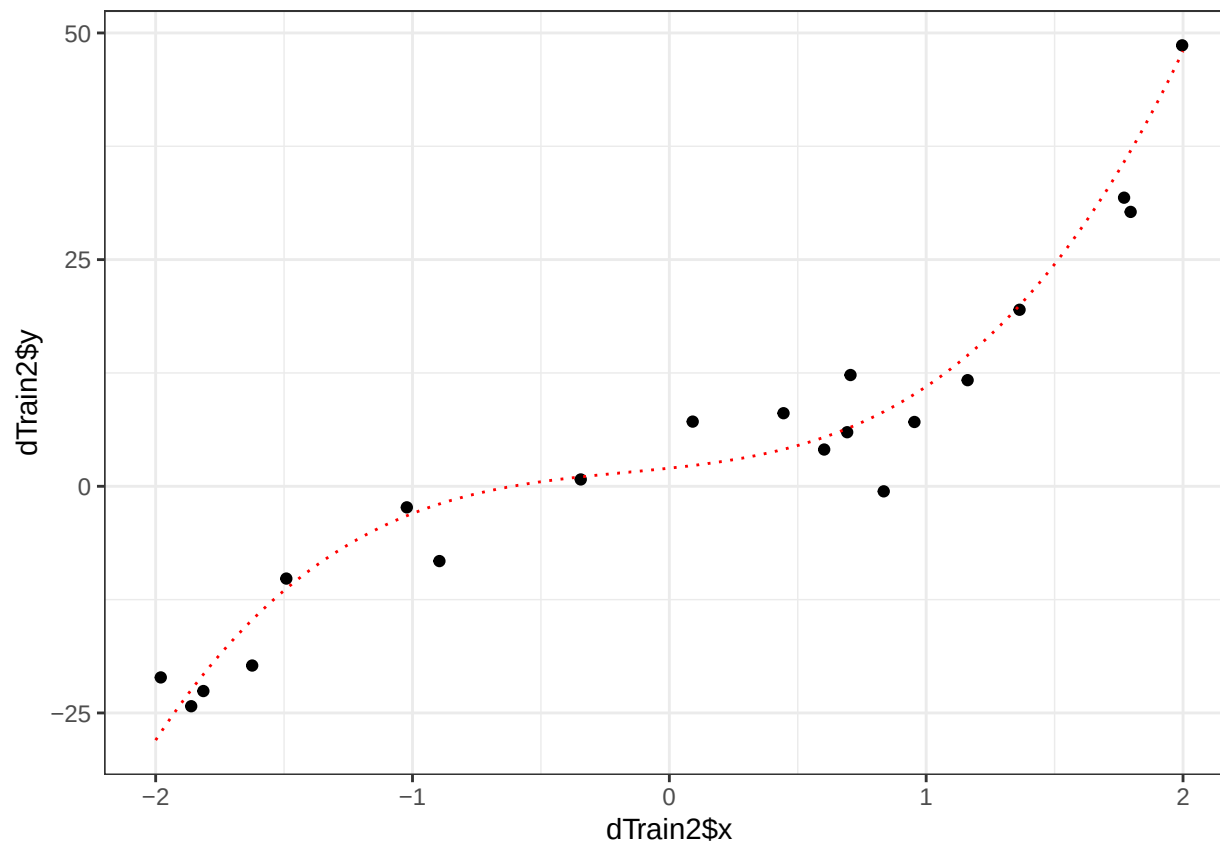
In this section we look at the accuracy of different models. With this aim we start by generating a new training data set *dTrain2* with 20 observations. The data is simulated from a cubic model given by $y = 2 + 3x + 2x^2 + 4x^3$, plus a normally distributed random variable with mean zero and variance 6. Note that apart from x and y we include in *dTrain2* columns for $x^2, x^3, \dots, x^7$ which are needed below.

```
dTrain2 <- tibble(x = runif(20, -2, 2), x2=x^2, x3=x^3, x4=x^4,
                  x5=x^5, x6=x^6, x7=x^7,
                  y = 2 + 3 * x + 2*x2+ 4*x3 + rnorm(length(x), 0, 6))
head(dTrain2, 5)
```

```
## # A tibble: 5 x 8
##       x      x2      x3      x4      x5      x6      x7      y
##   <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1  1.16  1.35  1.57  1.82  2.11  2.45  2.85  11.7
## 2  0.445 0.198 0.0879 0.0391 0.0174 0.00772 0.00343 8.06
## 3  0.835 0.697 0.582 0.485 0.405 0.338 0.282 -0.557
## 4 -1.02  1.04 -1.07  1.09 -1.12  1.14 -1.17 -2.32
## 5  0.954 0.910 0.869 0.829 0.791 0.755 0.720 7.09
```

Next, the training data set is shown in a scatter plot, together with the true relation between x and y (red dotted curve).

```
fun2 <- tibble(x = seq(from = -2, to = 2, by = 0.001),
               y = 2 + 3 * x + 2*x*x+ 4*x*x*x )
ggplot() +
  geom_point(aes(x = dTrain2$x, y = dTrain2$y)) +
  geom_line(aes(x = fun2$x, y = fun2$y), color="red", linetype="dotted")
```



Below we try to estimate the relation between x and y assuming in turn each of the following models:

linear $y = \beta_0 + \beta_1 x$, model *m2.1*

quadratic $y = \beta_0 + \beta_1 x + \beta_2 x^2$, model *m2.2*

cubic $y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3$, model *m2.3*

order 7 $y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \beta_4 x^4 + \beta_5 x^5 + \beta_6 x^6 + \beta_7 x^7$, model *m2.7*

We start by the linear model.

```
m2.1 <- lm(y ~ x , dTrain2)
summary(m2.1)
```

```
##
## Call:
## lm(formula = y ~ x, data = dTrain2)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -14.6351  -3.4597  -0.4629   3.0540  19.1284
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    2.996      1.644   1.822  0.0851 .
## x             13.277      1.257  10.561 3.83e-09 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##
## Residual standard error: 7.343 on 18 degrees of freedom
## Multiple R-squared:  0.8611, Adjusted R-squared:  0.8533
## F-statistic: 111.5 on 1 and 18 DF,  p-value: 3.828e-09
```

Then we generate a test data set *dTest2* with 30 observations.

```
dTest2 <- tibble(x = runif(30, -2, 2), x2=x^2, x3=x^3, x4=x^4,
                 x5=x^5, x6=x^6, x7=x^7,
                 y = 2 + 3 * x + 2*x2+ 4*x3 + rnorm(length(x), 0, 6))
```

We use the fitted linear model *m2.1* to predict the outcome value *y* for the test data.

```
dTest2$pred1 <- predict.lm(m2.1, newdata=dTest2)
```

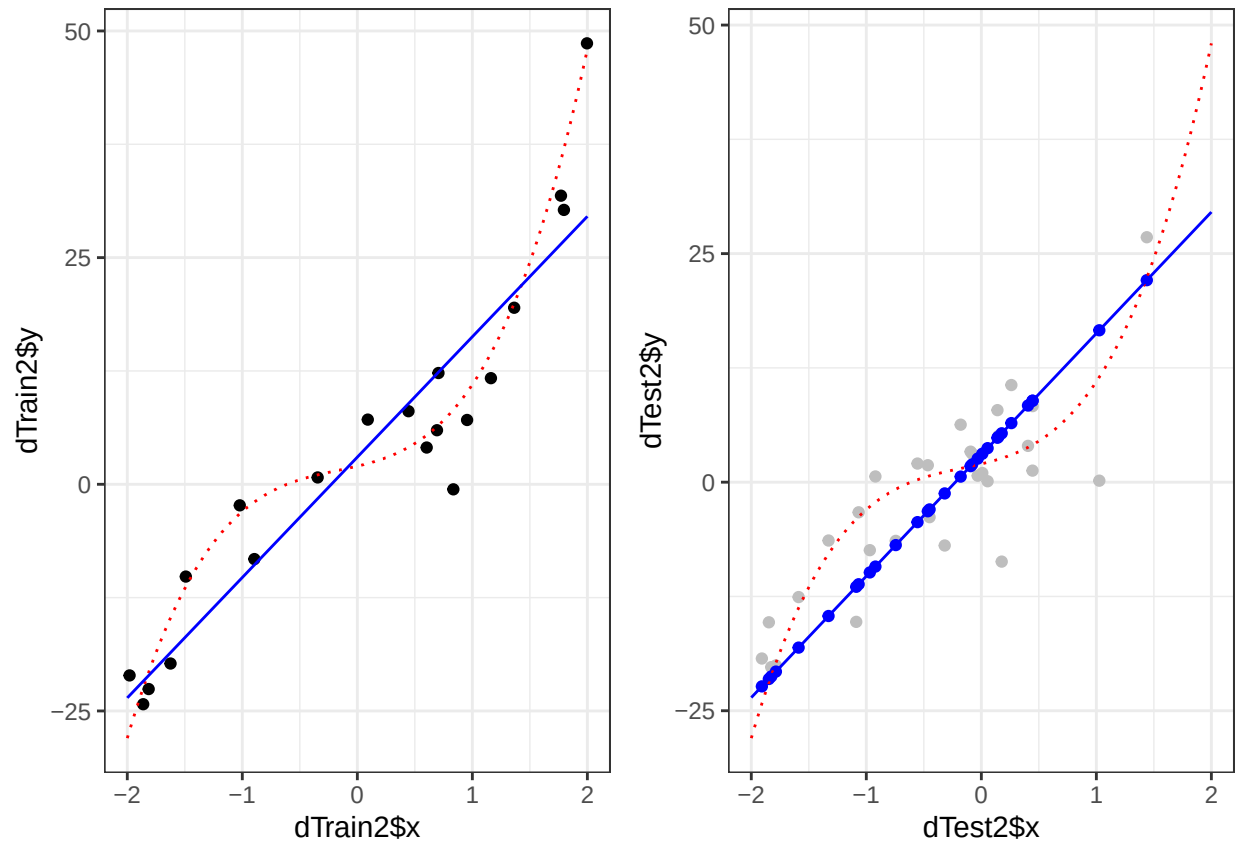
We plot the results for visualization. On the left side we show the scatter plot of the training data, the real relation between *x* and *y* (red dotted line) and the estimated relation between *x* and *y* (blue line). On the right side we show the scatter plot of the test data (gray dots), the predicted $\hat{y}$ values for this data (blue dots), the real relation between *x* and *y* (red dotted line) and the estimated relation between *x* and *y* (blue line).

```
xx=seq(from = -2, to = 2, by = 0.001)
estf <- tibble(x=xx, y1 = predict(m2.1, newdata=list(x=xx)) )

pp1 <- ggplot() +
  geom_point(aes(x = dTrain2$x, y= dTrain2$y),color="black") +
  geom_line(aes(x = fun2$x, y= fun2$y),color="red", linetype="dotted") +
  geom_line(aes(x = estf$x, y= estf$y1),color="blue")

pp2 <- ggplot() +
  geom_point(aes(x = dTest2$x, y= dTest2$y), color="gray")+
  geom_point(aes(x = dTest2$x, y= dTest2$pred1), color="blue")+
  geom_line(aes(x = fun2$x, y= fun2$y),color="red", linetype="dotted") +
  geom_line(aes(x = estf$x, y= estf$y1),color="blue")

grid.arrange(pp1, pp2, ncol=2)
```

We repeat the steps above but assuming a quadratic model.

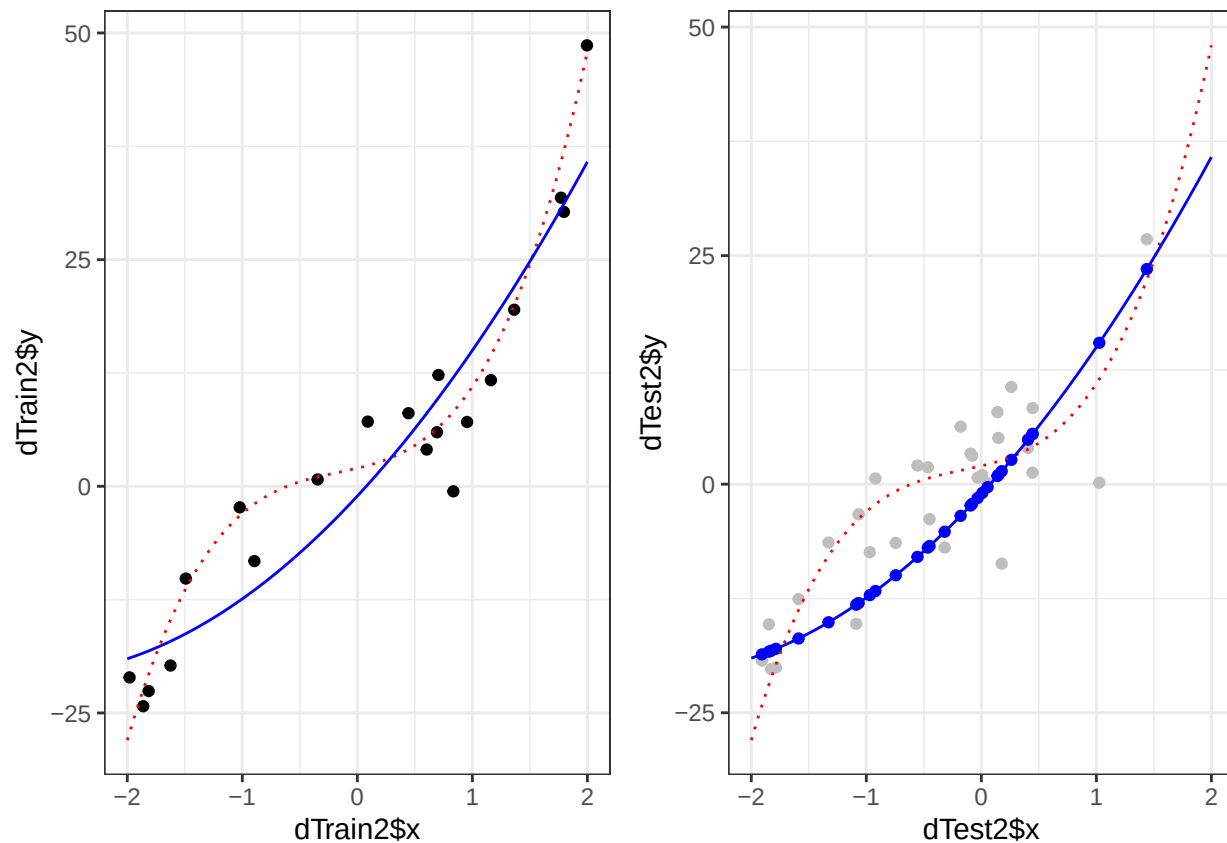
```
m2.2 <- lm(y ~ x + x2, dTrain2)
```

```
dTest2$pred2 <- predict.lm(m2.2, newdata= dTest2)
estf$y2 <- predict(m2.2, newdata=list(x=xx,x2=xx^2))
```

```
pp3 <- ggplot() +
  geom_point(aes(x = dTrain2$x, y= dTrain2$y),color="black") +
  geom_line(aes(x = fun2$x, y= fun2$y),color="red", linetype="dotted") +
  geom_line(aes(x = estf$x, y= estf$y2),color="blue")
```

```
pp4 <- ggplot() +
  geom_point(aes(x = dTest2$x, y= dTest2$y), color="gray")+
  geom_point(aes(x = dTest2$x, y= dTest2$pred2), color="blue")+
  geom_line(aes(x = fun2$x, y= fun2$y),color="red", linetype="dotted") +
  geom_line(aes(x = estf$x, y= estf$y2),color="blue")
```

```
grid.arrange(pp3, pp4, ncol=2)
```



We do the same for the cubic model.

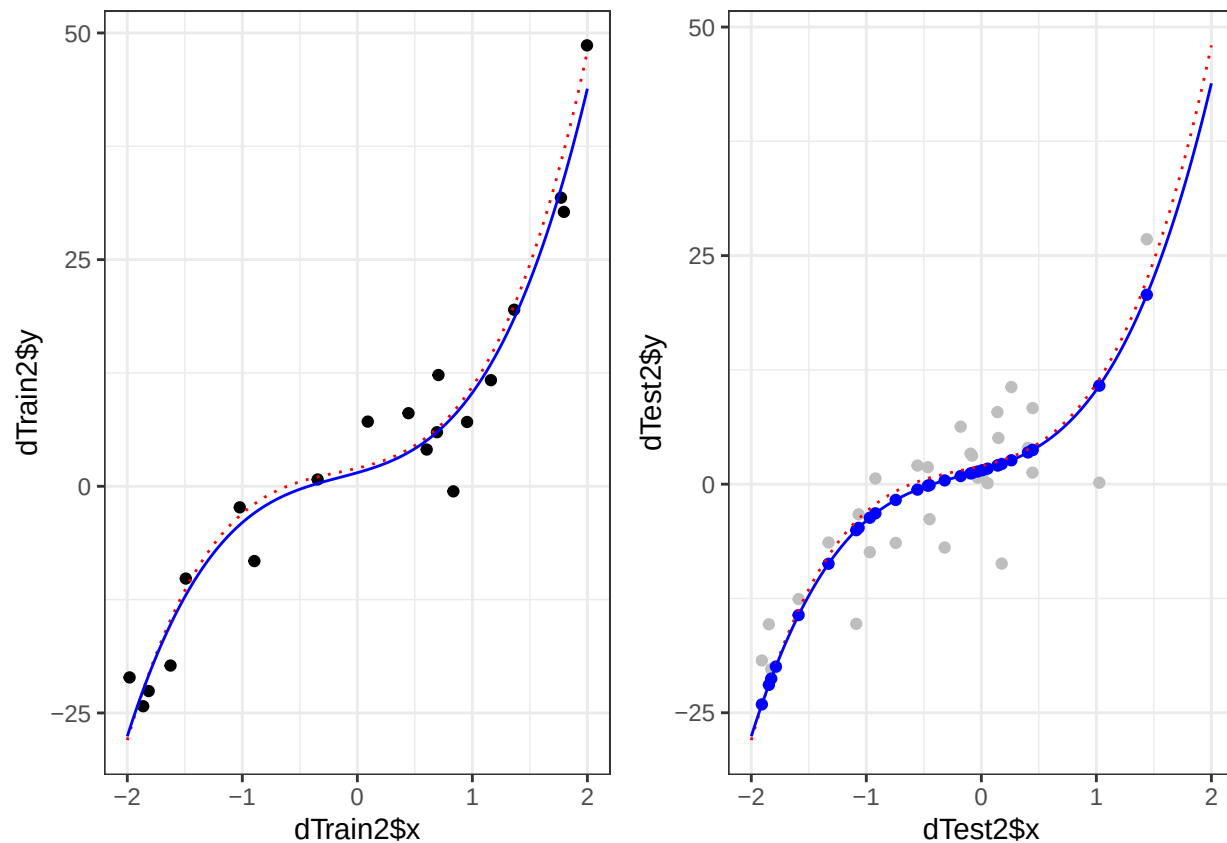
```
m2.3 <- lm(y ~ x + x2 + x3, dTrain2)
```

```
dTest2$pred3 <- predict.lm(m2.3, newdata=dTest2)
estf$y3 <- predict(m2.3, newdata=list(x=xx, x2=xx^2, x3=xx^3))
```

```
pp5 <- ggplot() +
  geom_point(aes(x = dTrain2$x, y = dTrain2$y), color="black") +
  geom_line(aes(x = fun2$x, y = fun2$y), color="red", linetype="dotted") +
  geom_line(aes(x = estf$x, y = estf$y3), color="blue")
```

```
pp6 <- ggplot() +
  geom_point(aes(x = dTest2$x, y = dTest2$y), color="gray") +
  geom_point(aes(x = dTest2$x, y = dTest2$pred3), color="blue") +
  geom_line(aes(x = fun2$x, y = fun2$y), color="red", linetype="dotted") +
  geom_line(aes(x = estf$x, y = estf$y3), color="blue")
```

```
grid.arrange(pp5, pp6, ncol=2)
```



And finally for the 7th order polynomial:

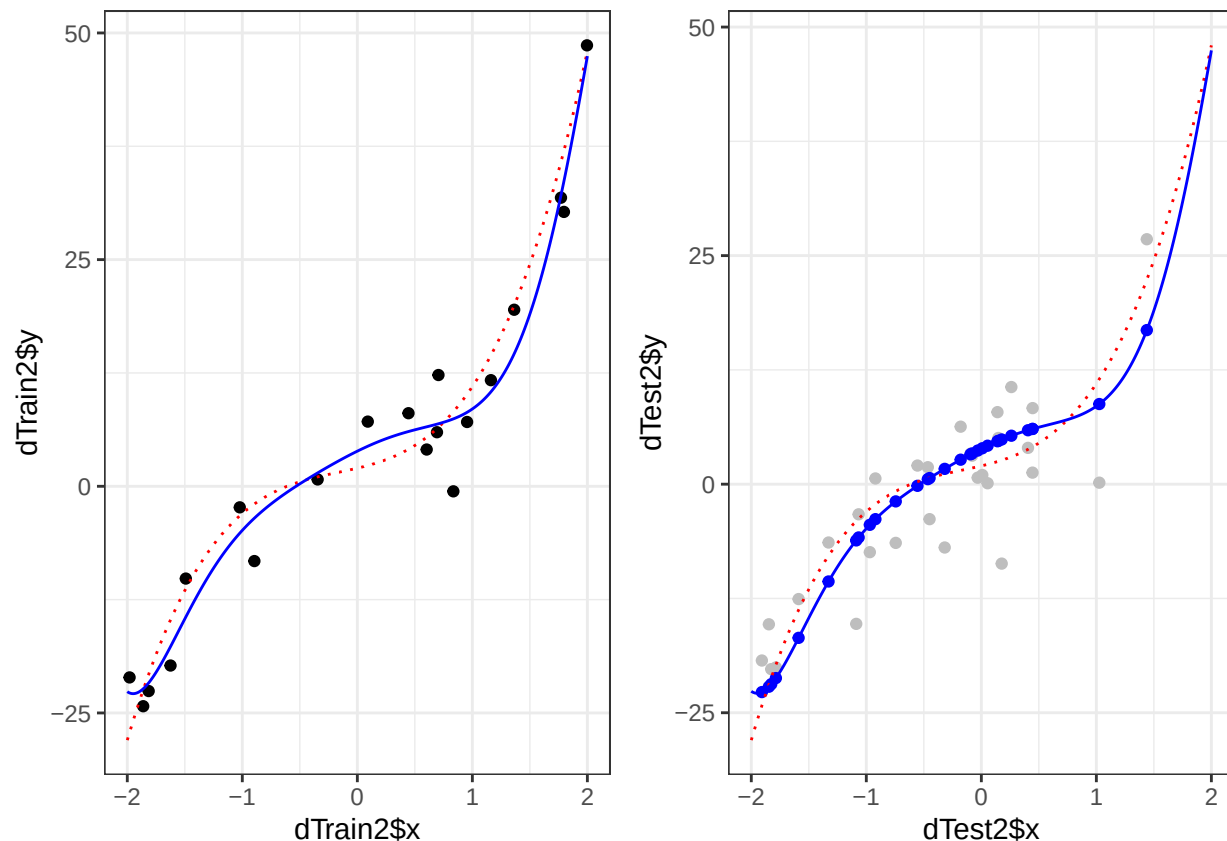
```
m2.7 <- lm(y ~ x + x2 + x3 + x4 + x5 + x6 + x7 , dTrain2)

dTest2$pred7 <- predict.lm(m2.7, newdata=dTest2)
estf$y7 <- predict(m2.7, newdata=list(x=xx,x2=xx^2,x3=xx^3,x4=xx^4,x5=xx^5,x6=xx^6,x7= xx^7))

pp7 <- ggplot() +
  geom_point(aes(x = dTrain2$x, y= dTrain2$y),color="black") +
  geom_line(aes(x = fun2$x, y= fun2$y),color="red", linetype="dotted") +
  geom_line(aes(x = estf$x, y= estf$y7),color="blue")

pp8 <- ggplot() +
  geom_point(aes(x = dTest2$x, y= dTest2$y), color="gray")+
  geom_point(aes(x = dTest2$x, y= dTest2$pred7), color="blue")+
  geom_line(aes(x = fun2$x, y= fun2$y),color="red", linetype="dotted") +
  geom_line(aes(x = estf$x, y= estf$y7),color="blue")

grid.arrange(pp7, pp8, ncol=2)
```



The accuracy of the models is estimated for the training and the test data sets using the MSE.

For m2.1 the training MSE is 48.52533 and the test MSE is 35.96567

For m2.2 the training MSE is 38.57942 and the test MSE is 41.74288

For m2.3 the training MSE is 14.92957 and the test MSE is 25.21621

For m2.7 the training MSE is 10.60078 and the test MSE is 27.11771

As the model complexity increases the fit to the training data gets better (training MSE is reduced). However the test MSE is worse for the most complex model. This is an indication of overfitting. The more complex models are fitting also the random noise in the data and hence generalize less well to the test data set.

Exercise

What would happen if we use more data (say 200 points) for both the training set and the testing set? Repeat the steps above using larger data sets and comment the results found.

write your solution here